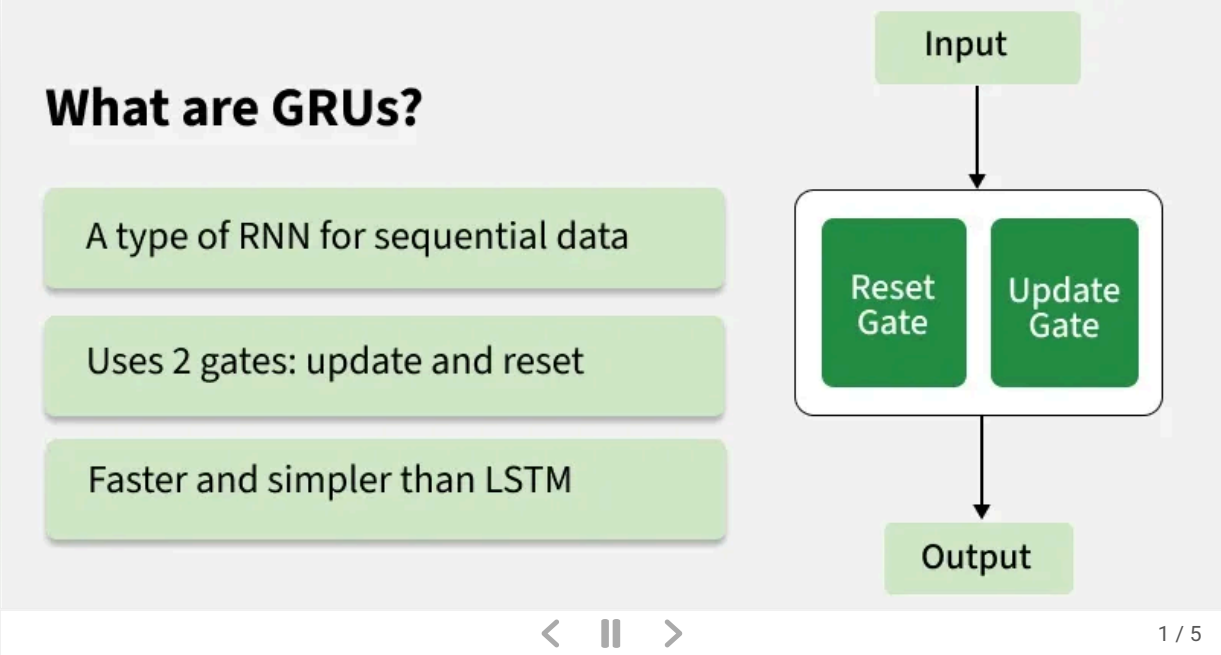
Gated Recurrent Unit Networks

Last Updated : 11 Jun, 2026

Gated Recurrent Unit (GRU) is a type of recurrent neural network designed for sequential data while reducing the complexity of traditional RNNs. GRUs are a simplified version of LSTMs that use update and reset gates to learn long term dependencies efficiently.

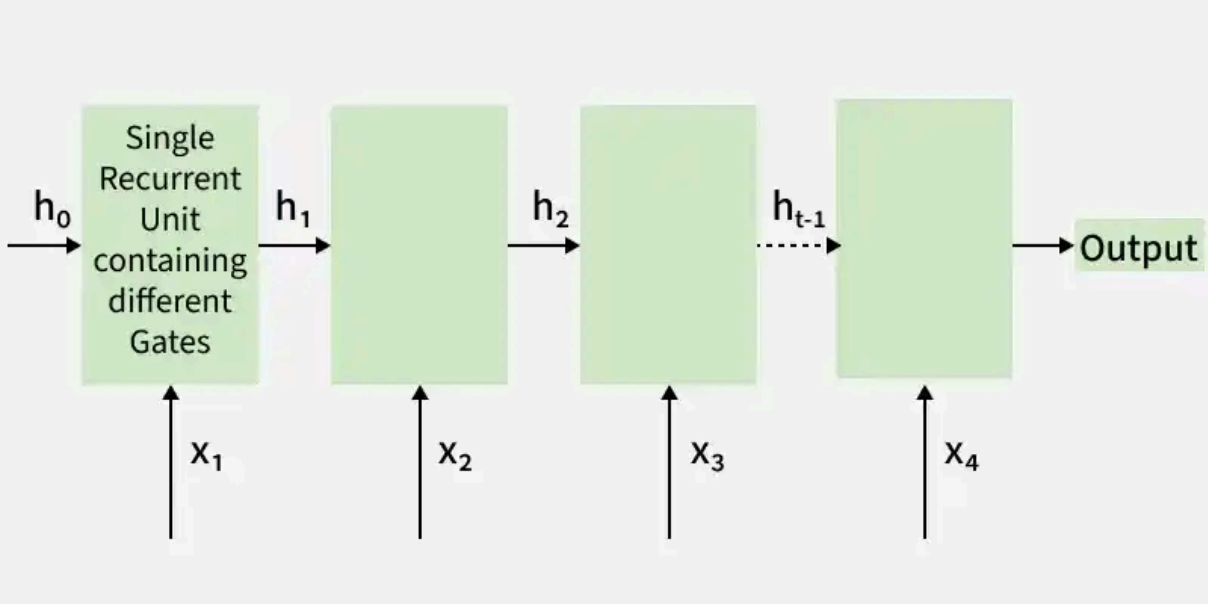
- Simplified alternative to LSTM
- Uses update and reset gates for information flow control
- Learns long-term dependencies with fewer parameters
- Handles sequence and time-series data effectively
- Widely used in NLP, speech processing and forecasting tasks



Gated Recurrent Units (GRU)

Gated Recurrent Units (GRUs) are a type of RNN introduced by Cho et al. in 2014. They use gating mechanisms to selectively retain important information and discard irrelevant details during sequence learning.

- Simplified version of LSTM architecture
- Uses two main gates: update gate and reset gate
- Efficiently learns long-term dependencies
- Reduces complexity compared to LSTMs
- Widely used for sequential and time-series data



Structure of GRU

The GRU consists of two main gates:

1. **Update Gate** (z_t): This gate decides how much information from previous hidden state should be retained for the next time step.
2. **Reset Gate** (r_t): This gate determines how much of the past hidden state should be forgotten.

These gates allow GRU to control the flow of information in a more efficient manner compared to traditional RNNs which solely rely on hidden state.

Equations for GRU Operations

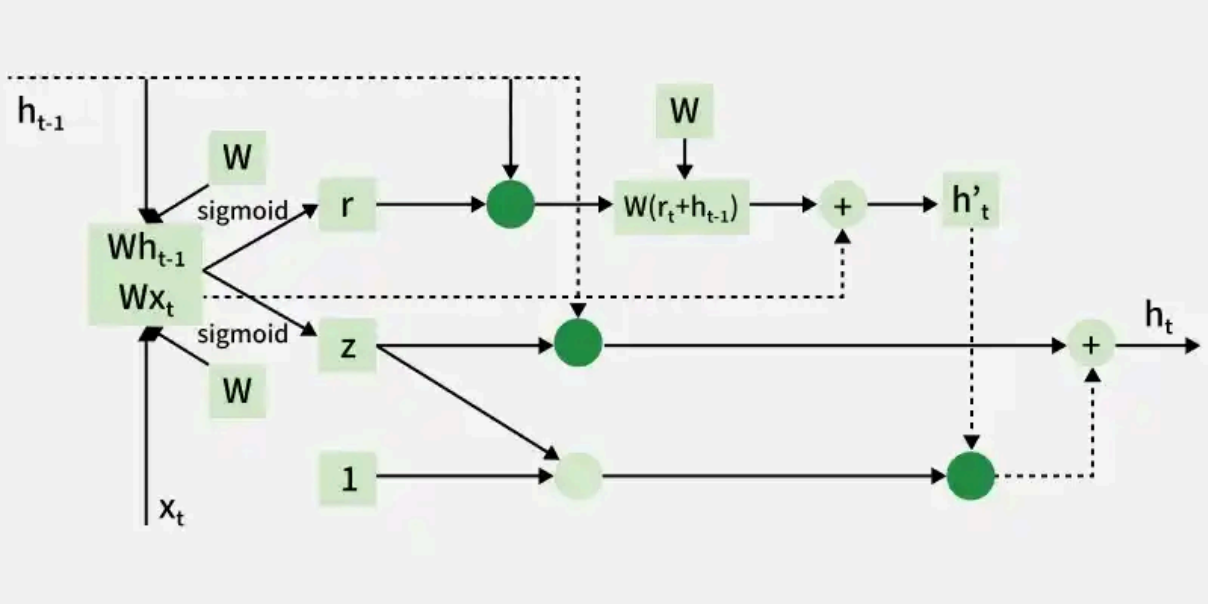
The internal workings of a GRU can be described using following equations

1. Reset gate:

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t] + b_r)$$

The reset gate controls how much of the previous hidden state is used when computing the candidate hidden state.

2. Update gate:



Update gate

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t] + b_z)$$

The update gate controls the balance between retaining the previous hidden state and incorporating the candidate hidden state.

3. Candidate hidden state:

$$h\prime_t = tanh(W_h \cdot [r_t \cdot h_{t-1}, x_t] + b_h)$$

This is the potential new hidden state calculated based on the current input and the previous hidden state.

4. Hidden state:

$$h_t = (1 - z_t) \cdot h_{t-1} + z_t \cdot h'_t$$

The final hidden state is a weighted average of the previous hidden state h_{t-1} and the candidate hidden state h'_t based on the update gate z_t.

Handling the Vanishing Gradient Problem

Like LSTMs, GRUs are designed to address the vanishing gradient problem commonly found in traditional RNNs.

- GRUs use gating mechanisms to regulate the flow of information and gradients during training
- These gates help preserve important information over long sequences
- They prevent gradients from shrinking too much, enabling better learning of long-term dependencies

GRU vs LSTM

Feature	LSTM (Long Short-Term Memory)	GRU (Gated Recurrent Unit)
Gates	3 (Input, Forget, Output)	2 (Update, Reset)
Cell State	Yes it has cell state	No (Hidden state only)
Training Speed	Slower due to complexity	Faster due to simpler architecture
Computational Load	Higher due to more gates and parameters	Lower due to fewer gates and parameters
Performance	Often better in tasks requiring long-term memory	Performs similarly in many tasks with less complexity

Implementation

Now let's implement simple GRU model in Python using **Keras**. We'll start by preparing the necessary libraries and dataset.

1. Importing Libraries

We will import the necessary libraries for implementing our GRU model such as **numpy**, **pandas**, **MinMaxScaler**, **TensorFlow** and **Adam**.

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import GRU, Dense
from tensorflow.keras.optimizers import Adam
```

2. Loading the Dataset

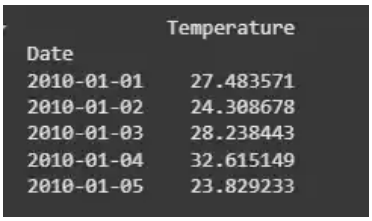
The dataset we're using is a time-series dataset containing daily temperature data i.e forecasting dataset. It spans 8,000 days starting from January 1, 2010.

*You can download dataset from **here**.*

- **pd.read_csv():** Reads a CSV file into a pandas DataFrame. Here, we are assuming that the dataset has a Date column which is set as the index of the DataFrame.
- **parse_dates=['Date']:** Ensures that the 'Date' column is automatically converted into datetime format.

```
df = pd.read_csv('data.csv', parse_dates=['Date'], index_col='Date')
print(df.head())
```

Output:



	Temperature
Date	
2010-01-01	27.483571
2010-01-02	24.308678
2010-01-03	28.238443
2010-01-04	32.615149
2010-01-05	23.829233

Loading the Dataset

3. Preprocessing the Data

The data is scaled using MinMaxScaler to normalize feature values between 0 and 1. Normalization helps neural networks train more effectively and prevents bias caused by features with larger values.

- Uses MinMaxScaler for normalization
- Scales features to the range 0–1
- Improves neural network training performance
- Prevents dominance of larger feature values

```
scaler = MinMaxScaler(feature_range=(0, 1))
scaled_data = scaler.fit_transform(df.values)
```

4. Preparing Data for GRU

We will define a function to prepare our data for training our model.

- **create_dataset():** Prepares the dataset for time-series forecasting. It creates sliding windows of time_step length to predict the next time step.
- **X.reshape()**: Reshapes the input data to fit the expected shape for the GRU which is 3D: i.e samples, time steps and features.

```
def create_dataset(data, time_step=1):
    X, y = [], []
    for i in range(len(data) - time_step - 1):
        X.append(data[i:(i + time_step), 0])
        y.append(data[i + time_step, 0])
    return np.array(X), np.array(y)


time_step = 100
X, y = create_dataset(scaled_data, time_step)
X = X.reshape(X.shape[0], X.shape[1], 1)
```

5. Building the GRU Model

We will define our GRU model with the following components:

- **GRU(units=50):** Adds a GRU layer with 50 units (neurons).
- **return_sequences=True:** Ensures that the GRU layer returns the entire sequence (required for stacking multiple GRU layers).
- **Dense(units=1):** The output layer which predicts a single value for the next time step.
- **Adam():** An adaptive optimizer commonly used in deep learning.

```
model = Sequential()
model.add(GRU(units=50, return_sequences=True, input_shape=(X.shape[1], 1)))
model.add(GRU(units=50))
model.add(Dense(units=1))
model.compile(optimizer=Adam(learning_rate=0.001), loss='mean_squared_error')
```

Output:

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, None, 64)	6,656
lstm (LSTM)	(None, None, 128)	98,816
dense (Dense)	(None, None, 104)	13,416

Total params: 118,888 (464.41 KB)
Trainable params: 118,888 (464.41 KB)
Non-trainable params: 0 (0.00 B)

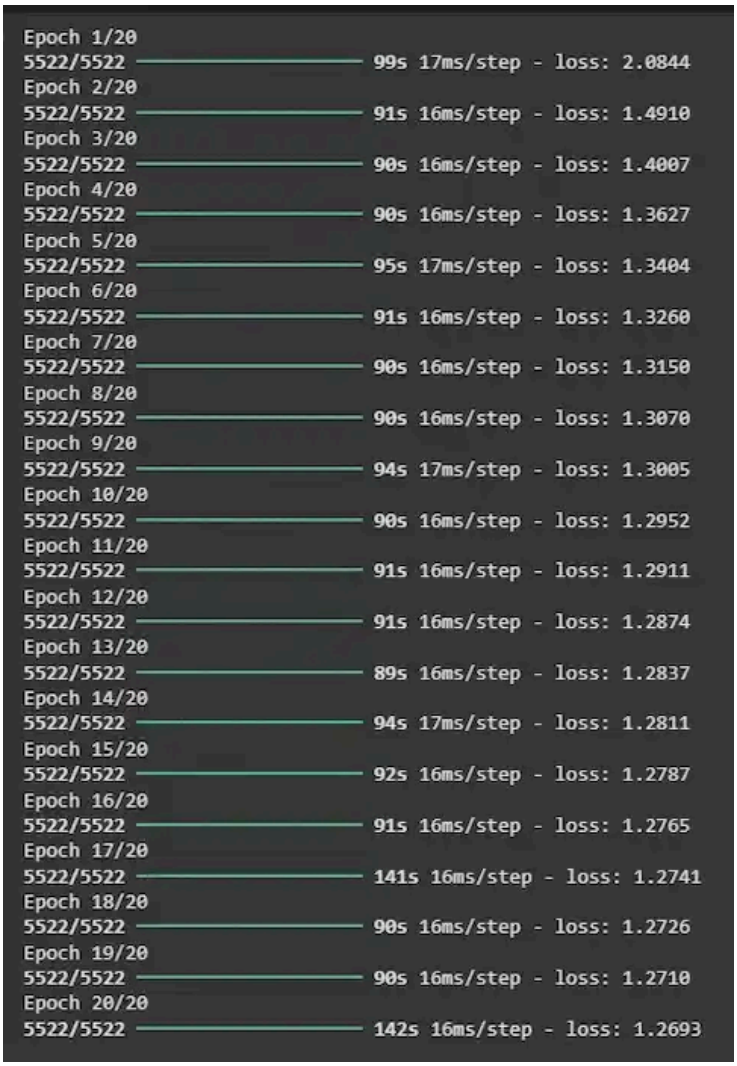
GRU model

6. Training the Model

model.fit() trains the model on the prepared dataset. The epochs=10 specifies the number of iterations over the entire dataset, and batch_size=32 defines the number of samples per batch.

```
model.fit(X, y, epochs=10, batch_size=32)
```

Output:



Training the model

7. Making Predictions

The trained GRU model is used to predict future values from the input sequence.

- Uses the last 100 scaled temperature values as input
- Reshapes input to (1, time_step, 1) for GRU compatibility
- samples = 1, time_steps = 100, and features = 1
- model.predict() generates predictions from the trained model

```
input_sequence = scaled_data[-time_step:].reshape(1, time_step, 1)
predicted_values = model.predict(input_sequence)
```

8. Inverse Transforming the Predictions

Inverse Transforming the Predictions refers to the process of converting the scaled (normalized) predictions back to their original scale.

- **scaler.inverse_transform():** Converts the normalized predictions back to their original scale.

```
predicted_values = scaler.inverse_transform(predicted_values)
```

```
print(
    f"The predicted temperature for the next day is: {predicted_values[0][0]:.2f}°C")
```

Output:

The predicted temperature for the next day is: 24.50°C

*Download full code from **here***

Applications

GRU networks are widely used for learning patterns from sequential and time-dependent data.

- Natural Language Processing (NLP) for translation and text generation
- Speech recognition and audio processing
- Time series forecasting such as weather and stock prediction
- Sentiment analysis and text classification
- Video and activity recognition tasks
- Recommendation systems and user behavior analysis

Suggested Quiz 5 Questions

What is the key difference between an LSTM and a GRU (Gated Recurrent Unit)?

A LSTMs are more complex and have more gates than GRUs

B GRUs require more training data

C GRUs are slower to train compared to LSTMs

Show More

Login to View Explanation

1/5 < Previous Next >

GeeksforGeeks 17

Explore

Machine Learning Basics

Python for Machine Learning

Feature Engineering

Supervised Learning

Unsupervised Learning

Model Evaluation and Tuning

Advanced Techniques

Machine Learning Practice

Courses

Corporate & Communications Address:

A-143, 6th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)

Registered Address:

K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305